

Contenido de esta parte

Antes de empezar

4.1 El problema: redundancia y anomalías

4.2 Primera forma normal (1FN)

4.3 Segunda forma normal (2FN)

4.4 Tercera forma normal (3FN)

4.5 Cómo reconocer el problema en una tabla real

4.6 Cuándo normalizar y cuándo desnormalizar

4.7 Errores habituales de diseño

Resumen de la Parte 4

Glosario de la Parte 4

Mini examen de la Parte 4

Parte 4. Normalización

Antes de empezar

En la Parte 3 aprendiste a construir tablas, relacionarlas y protegerlas. La normalización responde a una pregunta que viene antes: **¿están bien repartidos los datos entre las tablas?** Es el conjunto de reglas con el que se evita que los mismos datos se repitan y se contradigan.

La palabra asusta, pero la idea es muy sencilla: **cada dato debe vivir en un solo sitio**. Las «formas normales» son tres comprobaciones, de la más básica a la más fina, para saber si lo cumples.

Como en el resto del manual, cada punto tiene dos mitades: **primero sin código** (tablas dibujadas a mano, con datos de ejemplo) y **después con código** (el mismo razonamiento ejecutado en PostgreSQL, con capturas reales).

Al terminar serás capaz de:

- explicar qué problema resuelve la normalización y reconocer sus tres anomalías;
- aplicar la primera, segunda y tercera forma normal paso a paso;
- detectar dependencias entre columnas con consultas, y comprobar que no pierdes información al separar tablas;
- decidir cuándo conviene desnormalizar a propósito y cómo hacerlo sin riesgo;
- reconocer los errores de diseño más habituales.

Cómo se organiza cada punto

¿Qué es?, ¿para qué sirve?, ¿por qué lo necesito?, ¿cómo funciona?, primero sin código, paso a paso con código, código y resultado (con captura real), ejemplos, **profundizando**, ejercicio, solución, error habitual, buena práctica y comprobación.

El caso que usaremos en toda la parte

Partimos de una «hoja» de ventas de nuestra tienda, con **todos los datos en una sola tabla**. Cada fila es un producto de un pedido. A lo largo de la parte la iremos repartiendo en tablas bien diseñadas.

4.1 El problema: redundancia y anomalías

¿Qué es?

Redundancia es guardar el mismo dato en más de un sitio. Parece inofensivo («copio el nombre y así lo tengo a mano»), pero provoca tres problemas, llamados **anomalías**:

- **Anomalía de modificación:** cambias un dato en un sitio y olvidas los demás; los datos se contradicen.
- **Anomalía de inserción:** no puedes guardar un dato porque falta otro que aún no existe.
- **Anomalía de borrado:** al borrar una cosa pierdes sin querer datos de otra.

¿Para qué sirve conocerlas?

Para reconocer, al ver una tabla, si está mal repartida. Si puedes provocar alguna de las tres anomalías, la tabla necesita rediseño.

¿Por qué lo necesito?

Las anomalías no se notan al principio, con pocos datos y una sola persona. Aparecen con el uso: cuando hay cientos de filas y alguien cambia un email en un sitio y no en otro. Entonces ya hay datos corruptos, y corregirlos a mano es muy caro.

¿Cómo funciona?

Una tabla «plana» (con todo mezclado) repite en cada fila los datos del cliente, del producto y del pedido. Esa repetición es la fuente de las tres anomalías:

Anomalía	Causa	Ejemplo
Modificación	El mismo dato está en varias filas	Ana tiene dos emails distintos
Inserción	Una fila exige datos de varias cosas a la vez	No se puede guardar un producto sin un pedido
Borrado	Una fila guarda datos de varias cosas a la vez	Al borrar el pedido de Luis desaparece Luis

Primero, sin código: las tres anomalías a mano

1

Parte de la «hoja» de ventas: todo en una sola tabla

ventas_plano								
pedido	fecha	cliente	email	ciudad	provincia	producto	precio	cantidad
101	03-02	Ana García	ana@ejemplo.com	Alcalá	Madrid	Camiseta	19.95	2
101	03-02	Ana García	ana@ejemplo.com	Alcalá	Madrid	Gorra	9.90	1
102	03-05	Marta Ruiz	marta@ejemplo.com	Getafe	Madrid	Mochila	34.50	1
103	03-09	Ana García	ana@ejemplo.com	Alcalá	Madrid	Camiseta	19.95	1
104	03-10	Luis Pérez	luis@ejemplo.com	Sevilla	Sevilla	Gorra	9.90	3

En amarillo, datos de Ana que se repiten en cada fila. Cada vez que Ana compra, se vuelven a escribir su nombre, email, ciudad y provincia.

2

Problema 1: la modificación. Cambias el email de Ana en UNA fila y olvidas las demás

ventas_plano		
pedido	cliente	email
101	Ana García	ana.nueva@ejemplo.com
101	Ana García	ana@ejemplo.com
103	Ana García	ana@ejemplo.com

Ahora Ana tiene dos emails. ¿Cuál es el bueno? Los datos se contradicen.

3

Problema 2: la inserción. Quieres dar de alta un producto nuevo, pero aún nadie lo ha pedido

ventas_plano		
pedido	producto	precio
?	Taza	7.50

La tabla exige un pedido para cada fila: no puedes guardar el producto «Taza» hasta que alguien lo compre.

4

Problema 3: el borrado. Borrás el único pedido de Luis y desaparece también Luis

antes		después de borrar el pedido 104	
pedido	cliente	pedido	cliente
103	Ana García	103	Ana García
104	Luis Pérez		

Al borrar el pedido, perdiste los datos de Luis como cliente. Un pedido y un cliente no deberían depender uno del otro para existir.

Figura 4.1-a. Sin código: la tabla plana y sus tres anomalías.

Qué ves en la imagen. Paso 1: la tabla con todos los datos mezclados; en amarillo, lo que se repite de Ana. Paso 2: cambias el email de Ana en una sola fila (rojo) y queda con dos emails. Paso 3: intentas dar de alta un producto nuevo sin pedido y la tabla lo impide. Paso 4: al borrar el único pedido de Luis, Luis desaparece. **Compruébalo con tu dedo** sobre la figura antes de seguir.

Paso a paso: ahora con código

1. **Crea la tabla plana** con clave primaria (`pedido`, `producto`): cada pareja pedido-producto es una fila.
2. **Inserta cinco filas** de ejemplo.
3. **Lee la tabla** y busca a ojo lo que se repite.
4. **Provoca la anomalía de modificación:** cambia el email de Ana en una sola fila.
5. **Provoca la de inserción:** intenta guardar un producto sin pedido.
6. **Provoca la de borrado:** borra el pedido de Luis y mira si Luis sigue existiendo.

Código y resultado

```
CREATE TABLE ventas_plano (
    pedido    integer,
    fecha     date,
    cliente   text,
    email     text,
    ciudad    text,
    provincia text,
    producto  text,
    precio    numeric(10, 2),
    cantidad  integer,
    PRIMARY KEY (pedido, producto)
);

INSERT INTO ventas_plano VALUES
    (101, '2026-03-02', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid',
    'Camiseta', 19.95, 2),
    (101, '2026-03-02', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid',
    'Gorra', 9.90, 1),
    (102, '2026-03-05', 'Marta Ruiz', 'marta@ejemplo.com', 'Getafe', 'Madrid',
    'Mochila', 34.50, 1),
    (103, '2026-03-09', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid',
    'Camiseta', 19.95, 1),
    (104, '2026-03-10', 'Luis Pérez', 'luis@ejemplo.com', 'Sevilla',
    'Sevilla', 'Gorra', 9.90, 3);

SELECT * FROM ventas_plano ORDER BY pedido, producto;
```

psql — base de datos «normal»

```
normal=# CREATE TABLE ventas_plano (
normal-#   pedido    integer,
normal-#   fecha     date,
normal-#   cliente   text,
normal-#   email     text,
normal-#   ciudad    text,
normal-#   provincia text,
normal-#   producto  text,
normal-#   precio    numeric(10, 2),
normal-#   cantidad  integer,
normal-#   PRIMARY KEY (pedido, producto)
normal-# );
CREATE TABLE

normal=# INSERT INTO ventas_plano VALUES
normal-#   (101, '2026-03-02', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid', 'Camiseta', 19.95, 2),
normal-#   (101, '2026-03-02', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid', 'Gorra', 9.90, 1),
normal-#   (102, '2026-03-05', 'Marta Ruiz', 'marta@ejemplo.com', 'Getafe', 'Madrid', 'Mochila', 34.50, 1),
normal-#   (103, '2026-03-09', 'Ana García', 'ana@ejemplo.com', 'Alcalá', 'Madrid', 'Camiseta', 19.95, 1),
normal-#   (104, '2026-03-10', 'Luis Pérez', 'luis@ejemplo.com', 'Sevilla', 'Sevilla', 'Gorra', 9.90, 3);
INSERT 0 5

normal=# SELECT * FROM ventas_plano ORDER BY pedido, producto;
 pedido | fecha      | cliente | email           | ciudad | provincia | producto | precio | cantidad
-----+-----+-----+-----+-----+-----+-----+-----+-----
    101 | 2026-03-02 | Ana García | ana@ejemplo.com | Alcalá | Madrid   | Camiseta | 19.95 | 2
    101 | 2026-03-02 | Ana García | ana@ejemplo.com | Alcalá | Madrid   | Gorra    | 9.90  | 1
    102 | 2026-03-05 | Marta Ruiz | marta@ejemplo.com | Getafe | Madrid   | Mochila  | 34.50 | 1
    103 | 2026-03-09 | Ana García | ana@ejemplo.com | Alcalá | Madrid   | Camiseta | 19.95 | 1
    104 | 2026-03-10 | Luis Pérez | luis@ejemplo.com | Sevilla | Sevilla  | Gorra    | 9.90  | 3
(5 rows)
```

Figura 4.1-b. Captura real: la tabla plana con sus datos.

Qué ves en la imagen. `PRIMARY KEY (pedido, producto)` es una clave compuesta: una fila por cada producto de cada pedido. El `SELECT` muestra cinco filas. Fíjate en cuántas veces se repite «Ana García», su email, su ciudad y su provincia (tres veces cada uno), y en que «Camiseta» y «Gorra» repiten también su precio.

```
UPDATE ventas_plano
SET email = 'ana.nueva@ejemplo.com'
WHERE pedido = 101 AND producto = 'Camiseta';

SELECT DISTINCT cliente, email
FROM ventas_plano
WHERE cliente = 'Ana García';
```



```
psql — base de datos «normal»

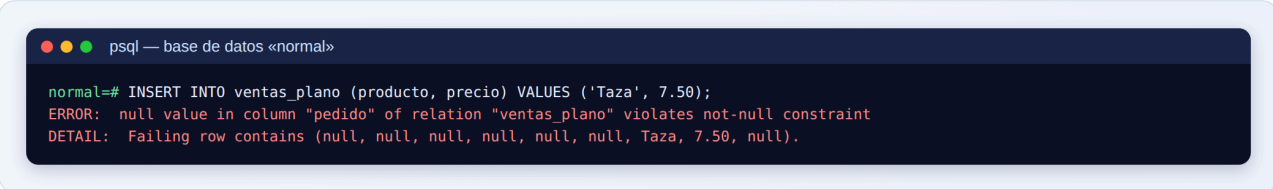
normal=# UPDATE ventas_plano
normal=# SET email = 'ana.nueva@ejemplo.com'
normal=# WHERE pedido = 101 AND producto = 'Camiseta';
UPDATE 1

normal=# SELECT DISTINCT cliente, email
normal=# FROM ventas_plano
normal=# WHERE cliente = 'Ana García';
 cliente | email
-----+-----
 Ana García | ana.nueva@ejemplo.com
 Ana García | ana@ejemplo.com
(2 rows)
```

Figura 4.1-c. Captura real: la anomalía de modificación. Ana acaba con dos emails.

Qué ves en la imagen. El `UPDATE` cambia el email solo en la fila del pedido 101 con la camiseta (`UPDATE 1`: una fila). La consulta con `DISTINCT` (sin repetidos) devuelve **dos** combinaciones para Ana García: el email nuevo y el antiguo. Los datos se contradicen, y la tabla no sabe cuál es el correcto.

```
INSERT INTO ventas_plano (producto, precio) VALUES ('Taza', 7.50);
```



```
psql — base de datos «normal»

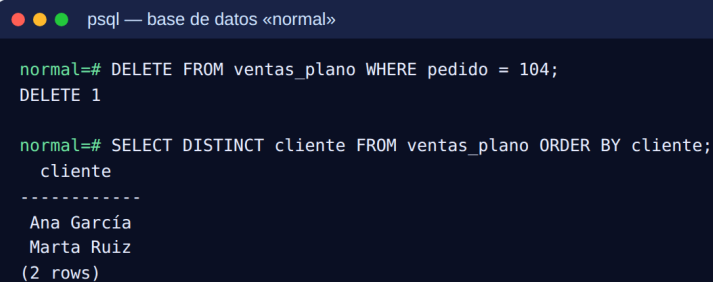
normal=# INSERT INTO ventas_plano (producto, precio) VALUES ('Taza', 7.50);
ERROR:  null value in column "pedido" of relation "ventas_plano" violates not-null constraint
DETAIL:  Failing row contains (null, null, null, null, null, null, Taza, 7.50, null).
```

Figura 4.1-d. Captura real: la anomalía de inserción. No se puede guardar un producto que nadie ha pedido.

Qué ves en la imagen. Intentamos guardar el producto «Taza» con su precio. La clave `(pedido, producto)` exige un pedido, y como no hay, el gestor responde `null value in column "pedido" ... violates not-null constraint`. El producto no puede existir en la base de datos hasta que alguien lo compre.

```
DELETE FROM ventas_plano WHERE pedido = 104;

SELECT DISTINCT cliente FROM ventas_plano ORDER BY cliente;
```



```
psql — base de datos «normal»

normal=# DELETE FROM ventas_plano WHERE pedido = 104;
DELETE 1

normal=# SELECT DISTINCT cliente FROM ventas_plano ORDER BY cliente;
 cliente
-----
Ana García
Marta Ruiz
(2 rows)
```

Figura 4.1-e. Captura real: la anomalía de borrado. Al borrar un pedido se pierde un cliente.

Qué ves en la imagen. `DELETE ... WHERE pedido = 104` borra una fila (`DELETE 1`). La consulta de clientes distintos devuelve solo Ana y Marta: **Luis ha desaparecido** de la base de datos, aunque era un cliente real, solo porque su único pedido se borró.

Ejemplo sencillo

Una agenda donde cada fila repite la dirección de la empresa. Si la empresa se muda, hay que cambiar muchas filas; si olvidas una, la empresa tiene dos direcciones.

Ejemplo real

Una academia guardaba en una sola tabla alumno, curso y profesor. Al cambiar el teléfono de un profesor, quedaron tres teléfonos distintos para la misma persona, y nadie sabía cuál era el vigente.

Profundizando

La redundancia no siempre es un error. A veces se repite un dato a propósito (punto 4.6). Lo grave es la redundancia **que nadie controla**.

Por qué el gestor no te avisa. Para PostgreSQL, «Ana García» con dos emails son filas válidas: él no sabe que describen a la misma persona. Las reglas de la normalización las pones tú con el diseño.

Las anomalías crecen con el tamaño. Con 5 filas son un fastidio; con 5 millones, un problema que requiere proyectos de limpieza de datos.

Ejercicio

1. En la tabla plana, ¿cuántas veces aparece el email de Ana antes de modificar nada? ¿Cuántas filas habría que tocar para cambiarlo bien?
2. Explica con tus palabras la anomalía de inserción de la figura 4.1-d.
3. ¿Cómo evitarías la anomalía de borrado con tablas separadas?

Solución

1. Aparece en 3 filas (pedido 101 dos veces y pedido 103). Habría que modificar las 3.
2. No se puede registrar un producto nuevo (con su precio) hasta que exista un pedido que lo contenga, porque la tabla mezcla productos y pedidos y exige datos de ambos.
3. Con una tabla `clientes` separada: un pedido borrado ya no arrastra al cliente, porque el cliente es una fila propia.

Error habitual

Pensar que repetir datos «hace las consultas más fáciles». Hace las lecturas algo más cómodas y las escrituras peligrosas.

```
-- Mal: el email del cliente repetido en cada fila de venta
UPDATE ventas_plano SET email = 'nuevo@ejemplo.com' WHERE pedido = 101 AND
producto = 'Camiseta';
```

Buena práctica

Antes de crear una tabla, pregúntate para cada columna: «¿este dato describe lo mismo que la fila, o describe otra cosa?». Si describe otra cosa, probablemente pertenece a otra tabla.

Comprobación

Para cualquier tabla tuya, intenta provocar las tres anomalías a mano: ¿puedo cambiar un dato en un sitio y olvidar otro? ¿Puedo guardar una cosa sin otra? ¿Puedo perder una cosa al borrar otra? Si respondes «sí» a alguna, hay que normalizar.

4.2 Primera forma normal (1FN)

¿Qué es?

Una tabla cumple la **primera forma normal (1FN)** cuando **cada celda contiene un solo valor** (un valor «atómico», que no se puede dividir con sentido) y **no hay grupos de columnas repetidas**.

¿Para qué sirve?

Para que cada dato se pueda buscar, contar, ordenar y enlazar por separado. Es la base de todas las demás formas normales.

¿Por qué lo necesito?

Con una lista dentro de una celda no puedes contar cuántas veces se vendió un producto, comprobar que el producto existe ni cambiar uno solo de los valores. Y con columnas repetidas (`producto1`, `producto2` ...) la tabla se queda pequeña o llena de huecos.

¿Cómo funciona?

Hay dos señales de que no se cumple la 1FN:

1. **Listas en una celda:** «Camiseta, Gorra».
2. **Columnas repetidas:** `telefono1`, `telefono2`, `telefono3`.

La solución es siempre la misma: **una fila por cada valor** (en la misma tabla, o en otra tabla enlazada si el valor es una cosa con vida propia).

Primero, sin código: de una lista a filas

1

Regla de la primera forma normal (1FN): una celda, un valor

Cada celda contiene **un solo dato**. No se permiten listas dentro de una celda ni columnas repetidas del mismo tipo.

Es la regla más básica: sin ella, ni siquiera se puede buscar bien.

2

Detecta el problema: listas en una celda

pedidos_lista (viola 1FN)		
pedido	cliente	productos
101	Ana García	Camiseta, Gorra
102	Marta Ruiz	Mochila
103	Ana García	Camiseta, Mochila

«Camiseta, Gorra» son dos datos en una celda.

3

Solución: una fila por cada valor de la lista

pedidos_1fn (cumple 1FN)		
pedido	cliente	producto
101	Ana García	Camiseta
101	Ana García	Gorra
102	Marta Ruiz	Mochila
103	Ana García	Camiseta
103	Ana García	Mochila

Ahora cada celda tiene un dato. Se puede contar, buscar y enlazar. (Aún quedan problemas: Ana se repite; eso lo resuelven las formas siguientes.)

Figura 4.2-a. Sin código: pasar una tabla con listas a primera forma normal.

Qué ves en la imagen. Paso 1: la regla. Paso 2: la tabla `pedidos_lista`, con dos celdas que contienen listas (en rojo). Paso 3: la solución, una fila por cada producto de cada pedido (en amarillo, las filas nuevas). Ahora cada celda tiene un único dato.

Paso a paso: ahora con código

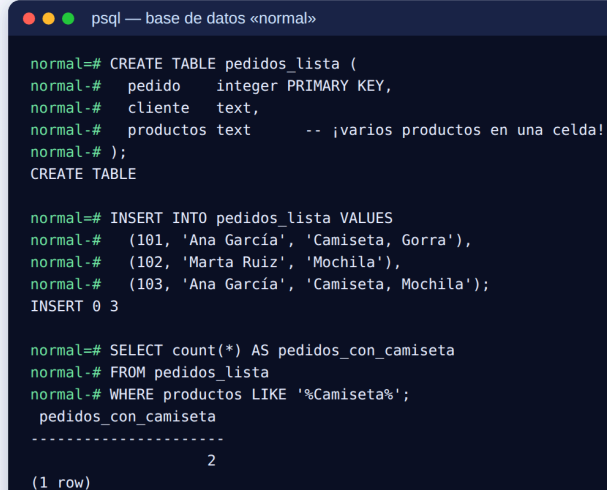
1. **Crea la tabla con la lista** y comprueba que no puedes contar bien.
2. **Descompón la lista en filas** con `string_to_array` (convierte el texto en una lista) y `unnest` (convierte una lista en filas).
3. **Guarda el resultado** en una tabla nueva con `CREATE TABLE ... AS SELECT`.
4. **Comprueba** que ahora sí se puede contar por producto.

Código y resultado

```
CREATE TABLE pedidos_lista (
  pedido    integer PRIMARY KEY,
  cliente   text,
  productos text      -- ¡varios productos en una celda!
);

INSERT INTO pedidos_lista VALUES
  (101, 'Ana García', 'Camiseta, Gorra'),
  (102, 'Marta Ruiz', 'Mochila'),
  (103, 'Ana García', 'Camiseta, Mochila');

SELECT count(*) AS pedidos_con_camiseta
FROM pedidos_lista
WHERE productos LIKE '%Camiseta%';
```



```
psql — base de datos «normal»

normal=# CREATE TABLE pedidos_lista (
normal-#   pedido    integer PRIMARY KEY,
normal-#   cliente   text,
normal-#   productos text      -- ¡varios productos en una celda!
normal-# );
CREATE TABLE

normal=# INSERT INTO pedidos_lista VALUES
normal-#   (101, 'Ana García', 'Camiseta, Gorra'),
normal-#   (102, 'Marta Ruiz', 'Mochila'),
normal-#   (103, 'Ana García', 'Camiseta, Mochila');
INSERT 0 3

normal=# SELECT count(*) AS pedidos_con_camiseta
normal-# FROM pedidos_lista
normal-# WHERE productos LIKE '%Camiseta%';
pedidos_con_camiseta
-----
2
(1 row)
```

Figura 4.2-b. Captura real: una tabla con listas en una celda, y la pregunta que no se puede responder bien.

Qué ves en la imagen. `productos text` guarda «Camiseta, Gorra» como un único texto. La consulta `LIKE '%Camiseta%'` cuenta los pedidos cuyo texto contiene esa palabra (2), pero no distingue «Camiseta» de «Camiseta térmica», ni cuenta unidades, ni garantiza que el producto exista.

```
SELECT pedido,
  cliente,
  unnest(string_to_array(productos, ', ')) AS producto
FROM pedidos_lista
ORDER BY pedido, producto;
```

```
psql — base de datos «normal»

normal=# SELECT pedido,
normal-#       cliente,
normal-#       unnest(string_to_array(productos, ', ')) AS producto
normal-# FROM pedidos_lista
normal-# ORDER BY pedido, producto;
 pedido | cliente | producto
-----+-----+-----
    101 | Ana García | Camiseta
    101 | Ana García | Gorra
    102 | Marta Ruiz | Mochila
    103 | Ana García | Camiseta
    103 | Ana García | Mochila
(5 rows)
```

Figura 4.2-c. Captura real: la lista descompuesta en filas.

Qué ves en la imagen. `string_to_array(productos, ', ')` parte el texto por cada coma y espacio; `unnest(...)` convierte esa lista en filas. El resultado son cinco filas, una por cada producto de cada pedido. Los valores ya son atómicos.

```
CREATE TABLE pedidos_lfn AS
SELECT pedido,
       cliente,
       unnest(string_to_array(productos, ', ')) AS producto
FROM pedidos_lista;

SELECT producto, count(*) AS pedidos
FROM pedidos_lfn
GROUP BY producto
ORDER BY producto;
```

```
psql — base de datos «normal»

normal=# CREATE TABLE pedidos_lfn AS
normal-# SELECT pedido,
normal-#       cliente,
normal-#       unnest(string_to_array(productos, ', ')) AS producto
normal-# FROM pedidos_lista;
normal-# SELECT 5

normal=# SELECT producto, count(*) AS pedidos
normal-# FROM pedidos_lfn
normal-# GROUP BY producto
normal-# ORDER BY producto;
 producto | pedidos
-----+-----
  Camiseta |      2
    Gorra  |      1
   Mochila |      2
(3 rows)
```

Figura 4.2-d. Captura real: guardar el resultado y contar por producto.

Qué ves en la imagen. `CREATE TABLE ... AS SELECT` crea una tabla nueva con el resultado de la consulta (`SELECT 5` indica que tiene cinco filas). Ahora `GROUP BY producto` agrupa y cuenta: Camiseta aparece en 2 pedidos, Gorra en 1 y Mochila en 2. Antes era imposible hacer esto con precisión.

Ejemplo sencillo

`telefonos = '600111, 910222'` pasa a ser dos filas en una tabla de teléfonos, una por número.

Ejemplo real

Un formulario guardaba las «aficiones» como texto separado por comas. Al querer enviar una oferta a los aficionados al senderismo, la búsqueda devolvía también «senderismo urbano» y «no senderismo». Pasar a una fila por afición lo resolvió.

Profundizando

¿Qué es «atómico»? Depende de lo que vayas a hacer con el dato. Un nombre completo en una columna es atómico si nunca vas a buscar por apellido; si vas a hacerlo, mejor separar nombre y apellidos. Una fecha es atómica aunque tenga día, mes y año, porque el gestor tiene tipos y funciones para trabajar con ella.

Arreglos y JSON. PostgreSQL permite guardar listas (*arrays*) y JSON dentro de una columna (Parte 6). Es útil en casos concretos, pero **rompe la 1FN a propósito**: solo conviene cuando no necesitas buscar ni enlazar los elementos por separado.

La 1FN no resuelve la redundancia. Tras la figura 4.2-d, Ana sigue repetida en cada fila. Eso lo resuelven las dos formas siguientes.

Ejercicio

- ¿Cumple la 1FN esta fila: `(1, 'Ana', 'Madrid, Alcalá')` ? ¿Y esta tabla: `(id, nombre, hijo1, hijo2, hijo3)` ?
- Convierte a 1FN la tabla `contactos(id, nombre, telefonos)` donde `telefonos` es una lista separada por comas.
- Escribe el SQL para obtener una fila por teléfono a partir de `contactos` (usa `string_to_array` y `unnest`).

Solución

- No cumple ninguna: la primera tiene una lista en la celda de ciudades; la segunda tiene columnas repetidas (`hijo1`, `hijo2`, `hijo3`).
- Una tabla `telefonos(contacto_id, numero)` con una fila por cada teléfono de cada contacto.
- Por ejemplo:

```
SELECT id AS contacto_id,
       unnest(string_to_array(telefonos, ',')) AS numero
FROM contactos;
```

Error habitual

Resolver las columnas repetidas añadiendo más columnas («hijo4», «hijo5»...), o las listas con separadores raros (; , |).

```
-- Mal: columnas repetidas
CREATE TABLE personas_mal (id integer PRIMARY KEY, nombre text, hijo1 text,
hijo2 text, hijo3 text);
-- Bien: una fila por hijo
CREATE TABLE hijos (persona_id integer, nombre text);
```

Buena práctica

Si ves un número en el nombre de una columna (`telefono1` , `producto2`), o un separador dentro de los datos, para: falta una fila o una tabla.

Comprobación

Para cada tabla pregúntate: ¿cada celda tiene un solo dato? ¿Hay columnas que se repiten con un número? Si la respuesta a la segunda es «sí», hay que pasar a filas.

4.3 Segunda forma normal (2FN)

¿Qué es?

Una tabla está en **segunda forma normal (2FN)** cuando cumple la 1FN y **cada dato depende de la clave completa**, no solo de una parte de ella. Solo tiene sentido cuando la clave primaria está formada por **varias columnas** (clave compuesta).

Una **dependencia** significa «el valor de A determina el valor de B»: si sé el producto, sé su precio. Se escribe **producto → precio**.

¿Para qué sirve?

Para que los datos que pertenecen a una sola parte de la clave (el precio es del producto, no de la pareja pedido-producto) vivan en su propia tabla y no se repitan en cada fila.

¿Por qué lo necesito?

En nuestra tabla plana, el precio de la camiseta se repite en cada pedido donde aparece. Si cambia su precio, hay que cambiarlo en muchas filas. Y la fecha del pedido se repite en cada línea del pedido.

¿Cómo funciona?

1. **Identifica la clave:** aquí **(pedido, producto)**.
2. **Para cada columna que no es clave, pregunta:** ¿depende de la clave entera, o solo de una parte?
3. **Las que dependen de una sola parte** se mueven a una tabla propia, con esa parte como clave.
4. **Las que dependen de la clave completa** se quedan.

En nuestro caso:

Columna	Depende de	¿Cumple 2FN?
fecha, cliente, email, ciudad, provincia	Solo del pedido	No
precio	Solo del producto	No
cantidad	De la pareja (pedido, producto)	Sí

Primero, sin código: separar por dependencias

1

Regla de la segunda forma normal (2FN): cada dato depende de TODA la clave

Si la clave está formada por **varias columnas**, cada dato debe depender de la clave completa, no solo de una parte.

Solo se aplica cuando la clave primaria es compuesta (aquí, pedido + producto).

2

Identifica la clave y de qué depende cada columna

ventas_plano · clave = (pedido, producto)

pedido	fecha	cliente	email	ciudad	provincia	producto	precio	cantidad
101	03-02	Ana García	ana@ejemplo.com	Alcalá	Madrid	Camiseta	19.95	2
101	03-02	Ana García	ana@ejemplo.com	Alcalá	Madrid	Gorra	9.90	1
102	03-05	Marta Ruiz	marta@ejemplo.com	Getafe	Madrid	Mochila	34.50	1

Azul: depende solo del pedido (fecha, cliente, email, ciudad, provincia). Naranja: depende solo del producto (precio). Verde: depende de la pareja completa (cantidad). Los azules y el naranja violan 2FN.

3

Separa lo que depende de una parte de la clave en su propia tabla

productos			pedidos						lineas		
id	nombre	precio	pedido	fecha	cliente	email	ciudad	provincia	pedido	producto_id	cantidad
1	Camiseta	19.95	101	03-02	Ana García	ana@ejemplo.com	Alcalá	Madrid	101	1	2
2	Gorra	9.90	102	03-05	Marta Ruiz	marta@ejemplo.com	Getafe	Madrid	101	2	1
3	Mochila	34.50							102	3	1

El precio vive una sola vez en productos. Los datos del pedido, una vez por pedido. Las líneas solo guardan lo que depende de la pareja: la cantidad.

Figura 4.3-a. Sin código: de qué depende cada columna y cómo se separa.

Qué ves en la imagen. Paso 2: cada columna coloreada según de qué depende (azul: solo del pedido; naranja: solo del producto; verde: de la pareja). Paso 3: el resultado: **productos** con el precio, **pedidos** con los datos del pedido y **lineas** con la cantidad, que es lo único que depende de la pareja.

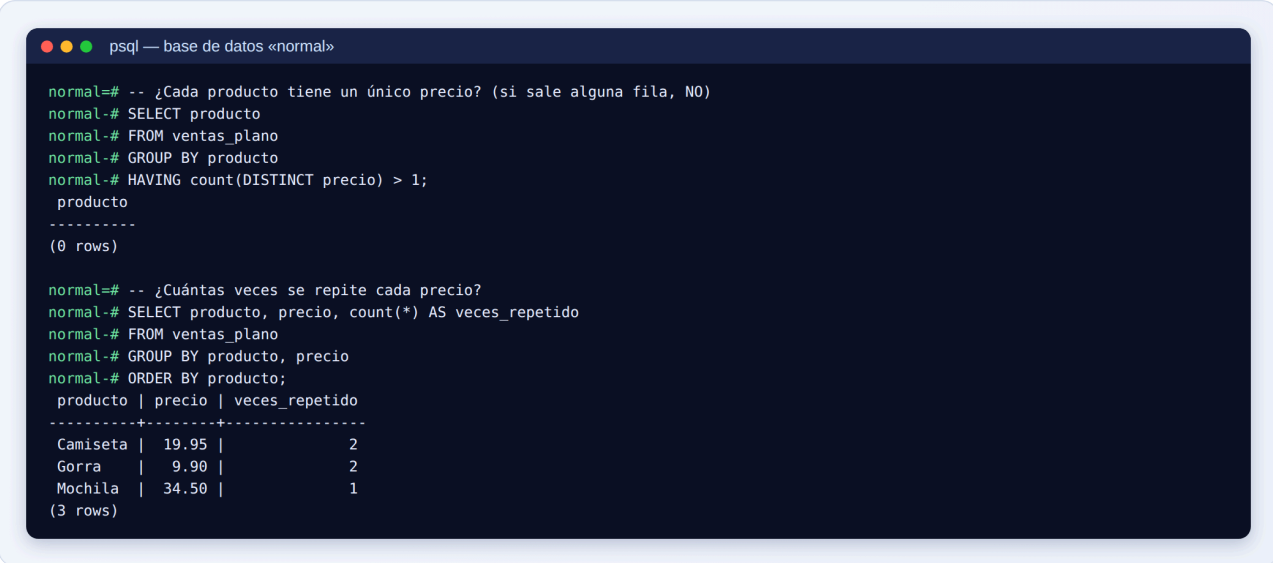
Paso a paso: ahora con código

1. **Comprueba la dependencia** **producto → precio** con una consulta: si ningún producto tiene dos precios distintos, la dependencia se cumple.
2. **Observa la repetición**: cuántas veces se repite cada precio.
3. **Crea la tabla** **productos** con los productos y precios distintos.
4. **Crea** **pedidos_2fn** con los datos que dependen del pedido.
5. **Crea** **lineas_2fn** con la pareja pedido-producto y la cantidad, enlazando con el **id** del producto.

Código y resultado

```
-- ¿Cada producto tiene un único precio? (si sale alguna fila, NO)
SELECT producto
FROM ventas_plano
GROUP BY producto
HAVING count(DISTINCT precio) > 1;

-- ¿Cuántas veces se repite cada precio?
SELECT producto, precio, count(*) AS veces_repetido
FROM ventas_plano
GROUP BY producto, precio
ORDER BY producto;
```



```
psql — base de datos «normal»

normal=# -- ¿Cada producto tiene un único precio? (si sale alguna fila, NO)
normal=# SELECT producto
normal=# FROM ventas_plano
normal=# GROUP BY producto
normal=# HAVING count(DISTINCT precio) > 1;
 producto
-----
(0 rows)

normal=# -- ¿Cuántas veces se repite cada precio?
normal=# SELECT producto, precio, count(*) AS veces_repetido
normal=# FROM ventas_plano
normal=# GROUP BY producto, precio
normal=# ORDER BY producto;
 producto | precio | veces_repetido
-----+-----+-----
 Camiseta | 19.95 |             2
   Gorra  |  9.90 |             2
  Mochila  | 34.50 |             1
(3 rows)
```

Figura 4.3-b. Captura real: comprobar que `producto → precio` y ver cuánto se repite.

Qué ves en la imagen. La primera consulta busca productos que tengan más de un precio distinto (`HAVING count(DISTINCT precio) > 1`): devuelve **0 filas**, es decir, cada producto tiene un único precio: la dependencia se cumple. La segunda muestra cuántas veces se repite cada precio: la camiseta y la gorra, dos veces cada una. Eso es redundancia: el mismo precio escrito varias veces.

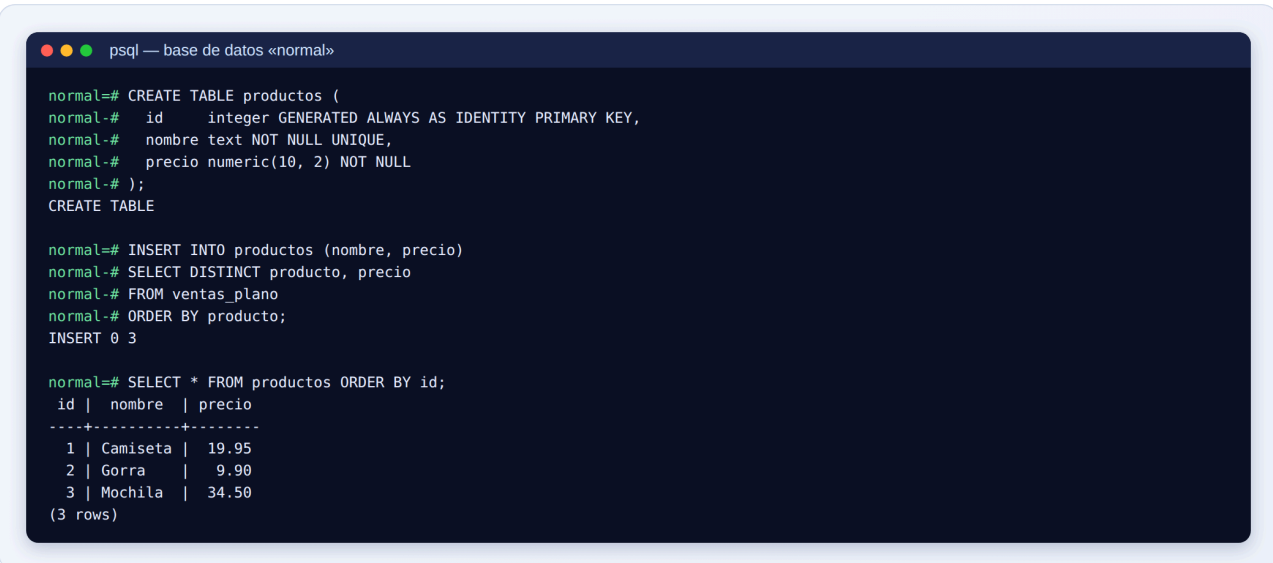
```

CREATE TABLE productos (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL UNIQUE,
  precio numeric(10, 2) NOT NULL
);

INSERT INTO productos (nombre, precio)
SELECT DISTINCT producto, precio
FROM ventas_plano
ORDER BY producto;

SELECT * FROM productos ORDER BY id;

```



```

psql — base de datos «normal»

normal=# CREATE TABLE productos (
normal-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
normal-#   nombre text NOT NULL UNIQUE,
normal-#   precio numeric(10, 2) NOT NULL
normal-# );
CREATE TABLE

normal=# INSERT INTO productos (nombre, precio)
normal-# SELECT DISTINCT producto, precio
normal-# FROM ventas_plano
normal-# ORDER BY producto;
INSERT 0 3

normal=# SELECT * FROM productos ORDER BY id;
 id | nombre | precio
-----+-----+-----
  1 | Camiseta | 19.95
  2 | Gorra   |  9.90
  3 | Mochila  | 34.50
(3 rows)

```

Figura 4.3-c. Captura real: la tabla `productos`, con cada precio una sola vez.

Qué ves en la imagen. `INSERT ... SELECT DISTINCT producto, precio FROM ventas_plano` copia a la tabla nueva los productos distintos con su precio (`INSERT 0 3`: tres filas). El `id` lo genera la identidad automática. Ahora el precio de cada producto está en un solo sitio.

```

CREATE TABLE pedidos_2fn AS
SELECT DISTINCT pedido, fecha, cliente, email, ciudad, provincia
FROM ventas_plano;

CREATE TABLE lineas_2fn AS
SELECT v.pedido, p.id AS producto_id, v.cantidad
FROM ventas_plano v
JOIN productos p ON p.nombre = v.producto;

SELECT * FROM pedidos_2fn ORDER BY pedido;

SELECT * FROM lineas_2fn ORDER BY pedido, producto_id;

```

```

psql — base de datos «normal»

normal=# CREATE TABLE pedidos_2fn AS
normal=# SELECT DISTINCT pedido, fecha, cliente, email, ciudad, provincia
normal=# FROM ventas_plano;
normal=# SELECT 4

normal=# CREATE TABLE lineas_2fn AS
normal=# SELECT v.pedido, p.id AS producto_id, v.cantidad
normal=# FROM ventas_plano v
normal=# JOIN productos p ON p.nombre = v.producto;
normal=# SELECT 5

normal=# SELECT * FROM pedidos_2fn ORDER BY pedido;
pedido | fecha      | cliente | email           | ciudad | provincia
-----+-----+-----+-----+-----+-----
 101   | 2026-03-02 | Ana García | ana@ejemplo.com | Alcalá | Madrid
 102   | 2026-03-05 | Marta Ruiz | marta@ejemplo.com | Getafe | Madrid
 103   | 2026-03-09 | Ana García | ana@ejemplo.com | Alcalá | Madrid
 104   | 2026-03-10 | Luis Pérez | luis@ejemplo.com | Sevilla | Sevilla
(4 rows)

normal=# SELECT * FROM lineas_2fn ORDER BY pedido, producto_id;
pedido | producto_id | cantidad
-----+-----+-----
 101   | 1           | 2
 101   | 2           | 1
 102   | 3           | 1
 103   | 1           | 1
 104   | 2           | 3
(5 rows)

```

Figura 4.3-d. Captura real: los datos del pedido y las líneas, separados.

Qué ves en la imagen. `pedidos_2fn` tiene una fila por pedido (cuatro, sin repetir fecha ni cliente por línea). `lineas_2fn` tiene una fila por cada producto de cada pedido (cinco), con la cantidad y el `producto_id` que apunta a `productos`. La unión con `JOIN ... ON p.nombre = v.producto` convierte el nombre del producto en su número.

Ejemplo sencillo

En una tabla de matrículas con clave `(alumno, curso)`, el nombre del curso depende solo del curso: va a la tabla de cursos.

Ejemplo real

Una tienda tenía la tabla `lineas(pedido, producto, nombre_producto, precio, cantidad)`. Al subir el precio de un producto hubo que actualizar miles de filas y se dejaron algunas sin actualizar. Separar `productos` lo resolvió.

Profundizando

La 2FN solo existe con claves compuestas. Si la clave primaria es una sola columna (por ejemplo, un `id`), la 2FN se cumple automáticamente.

Precio actual frente a precio vendido. Aquí hemos quitado `precio` de las líneas, pero en una tienda real conviene **guardar también el precio al que se vendió** (punto 4.6): no es redundancia accidental, es un dato histórico.

Aún queda trabajo. `pedidos_2fn` todavía repite los datos del cliente y de la ciudad. Es la tercera forma normal.

Ejercicio

1. En una tabla `notas(alumno, asignatura, nombre_asignatura, nota)` con clave `(alumno, asignatura)`, ¿qué columna viola la 2FN y por qué?
2. Escribe las dos tablas resultantes.
3. Comprueba con una consulta que `producto → precio` se cumple en `ventas_plano`.

Solución

1. `nombre_asignatura`: depende solo de `asignatura`, que es una parte de la clave.
2. `asignaturas(id, nombre)` y `notas(alumno, asignatura_id, nota)`.
3. `SELECT producto FROM ventas_plano GROUP BY producto HAVING count(DISTINCT precio) > 1;` devuelve 0 filas.

Error habitual

Aplicar la 2FN sobre una tabla con una clave de una sola columna (no hay nada que comprobar), o dejar en la tabla de líneas datos del producto «por comodidad».

```
-- Mal: el nombre del producto repetido en cada línea
CREATE TABLE lineas_mal (pedido_id integer, producto_id integer, nombre_producto
text, cantidad integer);
```

Buena práctica

Cuando una tabla tenga clave compuesta, revisa **cada columna no clave** con la pregunta: «¿cambiaría si cambiara solo una parte de la clave?».

Comprobación

Elige una tabla con clave compuesta. Para cada columna no clave, escribe de qué parte de la clave depende. Las que dependan de una sola parte se mueven a su tabla.

4.4 Tercera forma normal (3FN)

¿Qué es?

Una tabla está en **tercera forma normal (3FN)** cuando cumple la 2FN y **ningún dato depende de otro dato que no sea la clave**. Dicho de otro modo: todo dato depende de la clave, de **toda** la clave y de **nada más que** la clave.

Cuando A determina B y B determina C ($A \rightarrow B \rightarrow C$) se dice que C depende de A «a través de B»: una **dependencia transitiva**. La 3FN la elimina.

¿Para qué sirve?

Para que los datos que describen **otra cosa** (el cliente, la ciudad) no se mezclen con la tabla de la cosa principal (el pedido).

¿Por qué lo necesito?

En `pedidos_2fn`, cada pedido de Ana repite su email, su ciudad y su provincia. Si Ana se muda, hay que cambiar todos sus pedidos. La provincia, además, depende de la ciudad: «Alcalá → Madrid» se repite para todos los clientes de Alcalá.

¿Cómo funciona?

1. **Para cada columna no clave**, pregunta: ¿depende directamente de la clave o de otra columna no clave?
2. **Si depende de otra columna**, esa pareja de columnas forma una tabla propia, con la columna determinante como clave.
3. **Deja en la tabla original una clave extranjera** que apunte a la nueva.

En nuestro caso: `pedido → cliente → email, ciudad` y `ciudad → provincia`. Hay que crear `clientes` y `ciudades`.

Primero, sin código: separar las cadenas

1

Regla de la tercera forma normal (3FN): ningún dato depende de otro dato que no sea la clave

Un dato debe depender de la clave **directamente**, no «a través de» otro dato.

Se llama dependencia transitiva: $A \rightarrow B \rightarrow C$.

2

Busca cadenas de dependencia en pedidos

pedido $\rightarrow$ cliente $\rightarrow$ **email, ciudad**

pedido $\rightarrow$ cliente $\rightarrow$ ciudad $\rightarrow$ **provincia**

El email y la ciudad dependen del **cliente**, no del pedido. La provincia depende de la **ciudad**, no del cliente.

Si Ana hace 50 pedidos, su email, ciudad y provincia se repetirán 50 veces.

3

Separa cada cadena en su tabla

ciudades		
id	nombre	provincia
1	Sevilla	Sevilla
2	Getafe	Madrid
3	Alcalá	Madrid

clientes			
id	nombre	email	ciudad_id
1	Luis Pérez	luis@ejemplo.com	1
2	Ana García	ana@ejemplo.com	3
3	Marta Ruiz	marta@ejemplo.com	2

pedidos		
id	cliente_id	fecha
101	2	03-02
102	3	03-05
103	2	03-09
104	1	03-10

Cada dato vive en un solo sitio. Ana cambia de email en una sola fila. Las tablas se enlazan con números (claves extranjeras).

4

Comprueba: ¿se puede reconstruir la tabla original?

Sí: uniendo líneas, pedidos, clientes, ciudades y productos se obtiene la hoja original, **sin perder ni inventar ninguna fila**. La normalización no pierde información: la reorganiza.

En el apartado de código lo demostraremos con una consulta que compara ambas tablas.

Figura 4.4-a. Sin código: encontrar las cadenas de dependencia y separarlas en tablas.

Qué ves en la imagen. Paso 2: las cadenas pedido $\rightarrow$ cliente $\rightarrow$ email/ciudad y ciudad $\rightarrow$ provincia. Paso 3: se crean **ciudades** y **clientes**, y **pedidos** guarda solo el número de cliente (en amarillo, los números de enlace). Paso 4: la comprobación de que la tabla original se puede reconstruir sin perder ni inventar filas.

Paso a paso: ahora con código

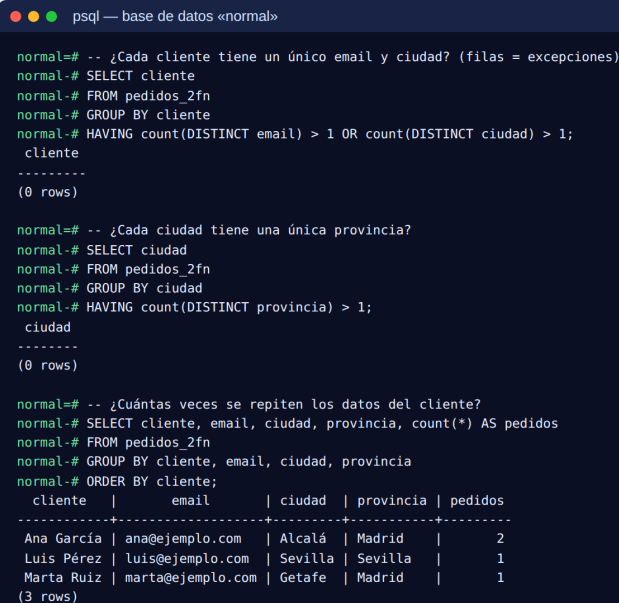
1. **Comprueba las dependencias:** ¿cada cliente tiene un solo email y ciudad? ¿cada ciudad una sola provincia?
2. **Crea las tablas definitivas:** **ciudades**, **clientes**, **pedidos** y **lineas_pedido**, con claves y relaciones.
3. **Rellénalas desde los datos existentes** con **INSERT ... SELECT**, en el orden correcto (primero las que no dependen de otras).
4. **Reconstruye la tabla original con una vista** y compara con la original usando **EXCEPT**: si no sobra ni falta ninguna fila, no has perdido información.
5. **Repite las tres anomalías** y comprueba que ya no ocurren.

Código y resultado

```
-- ¿Cada cliente tiene un único email y ciudad? (filas = excepciones)
SELECT cliente
FROM pedidos_2fn
GROUP BY cliente
HAVING count(DISTINCT email) > 1 OR count(DISTINCT ciudad) > 1;

-- ¿Cada ciudad tiene una única provincia?
SELECT ciudad
FROM pedidos_2fn
GROUP BY ciudad
HAVING count(DISTINCT provincia) > 1;

-- ¿Cuántas veces se repiten los datos del cliente?
SELECT cliente, email, ciudad, provincia, count(*) AS pedidos
FROM pedidos_2fn
GROUP BY cliente, email, ciudad, provincia
ORDER BY cliente;
```



```
psql — base de datos «normal»

normal=# -- ¿Cada cliente tiene un único email y ciudad? (filas = excepciones)
normal=# SELECT cliente
normal=# FROM pedidos_2fn
normal=# GROUP BY cliente
normal=# HAVING count(DISTINCT email) > 1 OR count(DISTINCT ciudad) > 1;
 cliente
-----
(0 rows)

normal=# -- ¿Cada ciudad tiene una única provincia?
normal=# SELECT ciudad
normal=# FROM pedidos_2fn
normal=# GROUP BY ciudad
normal=# HAVING count(DISTINCT provincia) > 1;
 ciudad
-----
(0 rows)

normal=# -- ¿Cuántas veces se repiten los datos del cliente?
normal=# SELECT cliente, email, ciudad, provincia, count(*) AS pedidos
normal=# FROM pedidos_2fn
normal=# GROUP BY cliente, email, ciudad, provincia
normal=# ORDER BY cliente;
 cliente | email | ciudad | provincia | pedidos
-----+-----+-----+-----+-----
 Ana García | ana@ejemplo.com | Alcalá | Madrid | 2
 Luis Pérez | luis@ejemplo.com | Sevilla | Sevilla | 1
 Marta Ruiz | marta@ejemplo.com | Getafe | Madrid | 1
(3 rows)
```

Figura 4.4-b. Captura real: comprobar las dependencias de cliente y ciudad.

Qué ves en la imagen. Las dos primeras consultas devuelven 0 filas: cada cliente tiene un solo email y ciudad, y cada ciudad una sola provincia (las dependencias se cumplen). La tercera muestra la repetición: los datos de Ana (email, ciudad, provincia) están escritos tantas veces como pedidos tiene.

```

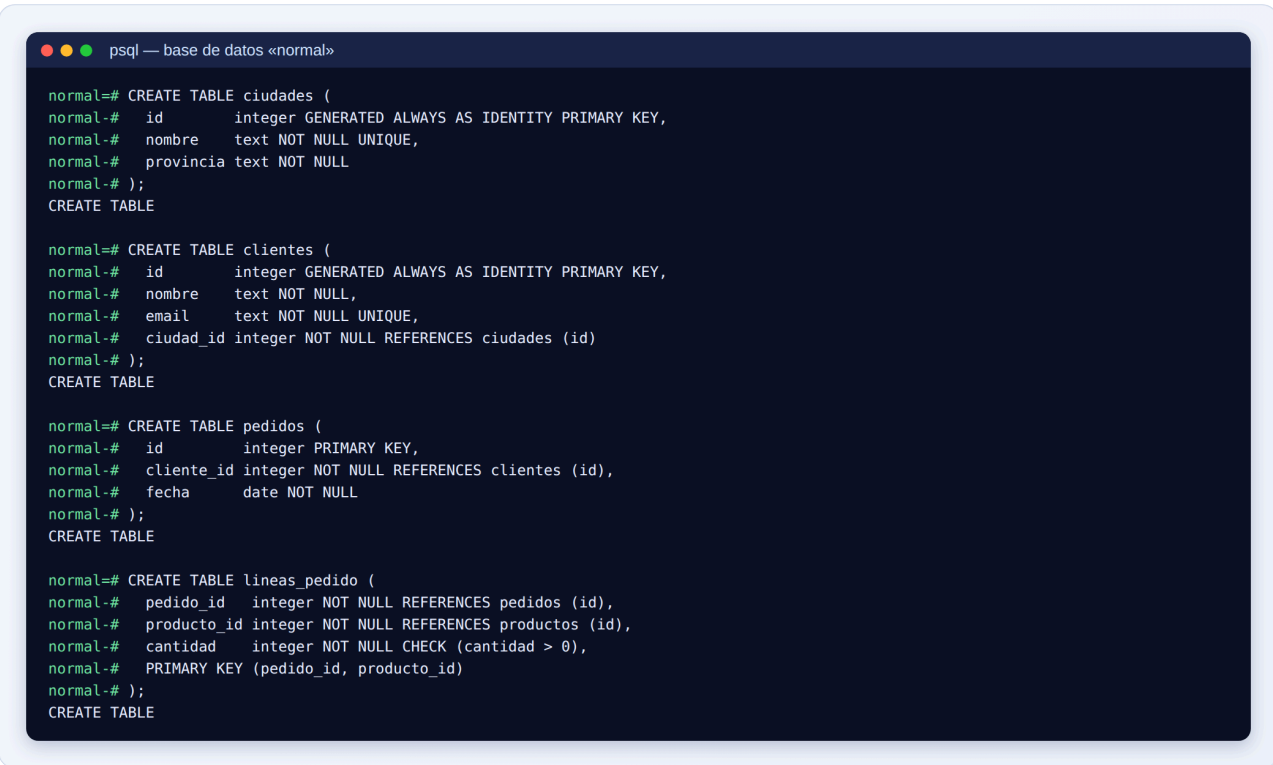
CREATE TABLE ciudades (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL UNIQUE,
  provincia   text NOT NULL
);

CREATE TABLE clientes (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  email       text NOT NULL UNIQUE,
  ciudad_id   integer NOT NULL REFERENCES ciudades (id)
);

CREATE TABLE pedidos (
  id          integer PRIMARY KEY,
  cliente_id  integer NOT NULL REFERENCES clientes (id),
  fecha       date NOT NULL
);

CREATE TABLE lineas_pedido (
  pedido_id   integer NOT NULL REFERENCES pedidos (id),
  producto_id integer NOT NULL REFERENCES productos (id),
  cantidad    integer NOT NULL CHECK (cantidad > 0),
  PRIMARY KEY (pedido_id, producto_id)
);

```



```

psql — base de datos «normal»

normal=# CREATE TABLE ciudades (
normal-#  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
normal-#  nombre      text NOT NULL UNIQUE,
normal-#  provincia   text NOT NULL
normal-# );
CREATE TABLE

normal=# CREATE TABLE clientes (
normal-#  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
normal-#  nombre      text NOT NULL,
normal-#  email       text NOT NULL UNIQUE,
normal-#  ciudad_id   integer NOT NULL REFERENCES ciudades (id)
normal-# );
CREATE TABLE

normal=# CREATE TABLE pedidos (
normal-#  id          integer PRIMARY KEY,
normal-#  cliente_id  integer NOT NULL REFERENCES clientes (id),
normal-#  fecha       date NOT NULL
normal-# );
CREATE TABLE

normal=# CREATE TABLE lineas_pedido (
normal-#  pedido_id   integer NOT NULL REFERENCES pedidos (id),
normal-#  producto_id integer NOT NULL REFERENCES productos (id),
normal-#  cantidad    integer NOT NULL CHECK (cantidad > 0),
normal-#  PRIMARY KEY (pedido_id, producto_id)
normal-# );
CREATE TABLE

```

Figura 4.4-c. Captura real: las tablas definitivas, con claves y relaciones.

Qué ves en la imagen. `ciudades` guarda cada ciudad con su provincia una sola vez. `clientes` apunta a su ciudad con `ciudad_id`. `pedidos` apunta a su cliente con `cliente_id`. `lineas_pedido` apunta a pedidos y productos, con clave compuesta. Cada dato tiene su sitio.

```
INSERT INTO ciudades (nombre, provincia)
SELECT DISTINCT ciudad, provincia FROM pedidos_2fn;

INSERT INTO clientes (nombre, email, ciudad_id)
SELECT DISTINCT p.cliente, p.email, c.id
FROM pedidos_2fn p
JOIN ciudades c ON c.nombre = p.ciudad;

INSERT INTO pedidos (id, cliente_id, fecha)
SELECT p.pedido, c.id, p.fecha
FROM pedidos_2fn p
JOIN clientes c ON c.email = p.email;

INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad)
SELECT pedido, producto_id, cantidad FROM lineas_2fn;

SELECT * FROM ciudades ORDER BY id;

SELECT * FROM clientes ORDER BY id;
```

```
psql — base de datos «normal»

normal=# INSERT INTO ciudades (nombre, provincia)
normal=# SELECT DISTINCT ciudad, provincia FROM pedidos_2fn;
INSERT 0 3

normal=# INSERT INTO clientes (nombre, email, ciudad_id)
normal=# SELECT DISTINCT p.cliente, p.email, c.id
normal=# FROM pedidos_2fn p
normal=# JOIN ciudades c ON c.nombre = p.ciudad;
INSERT 0 3

normal=# INSERT INTO pedidos (id, cliente_id, fecha)
normal=# SELECT p.pedido, c.id, p.fecha
normal=# FROM pedidos_2fn p
normal=# JOIN clientes c ON c.email = p.email;
INSERT 0 4

normal=# INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad)
normal=# SELECT pedido, producto_id, cantidad FROM lineas_2fn;
INSERT 0 5

normal=# SELECT * FROM ciudades ORDER BY id;
 id | nombre | provincia
-----+-----+-----
  1 | Sevilla | Sevilla
  2 | Getafe  | Madrid
  3 | Alcalá  | Madrid
(3 rows)

normal=# SELECT * FROM clientes ORDER BY id;
 id | nombre | email | ciudad_id
-----+-----+-----+-----
  1 | Luis Pérez | luis@ejemplo.com | 1
  2 | Ana García | ana@ejemplo.com | 3
  3 | Marta Ruiz | marta@ejemplo.com | 2
(3 rows)
```

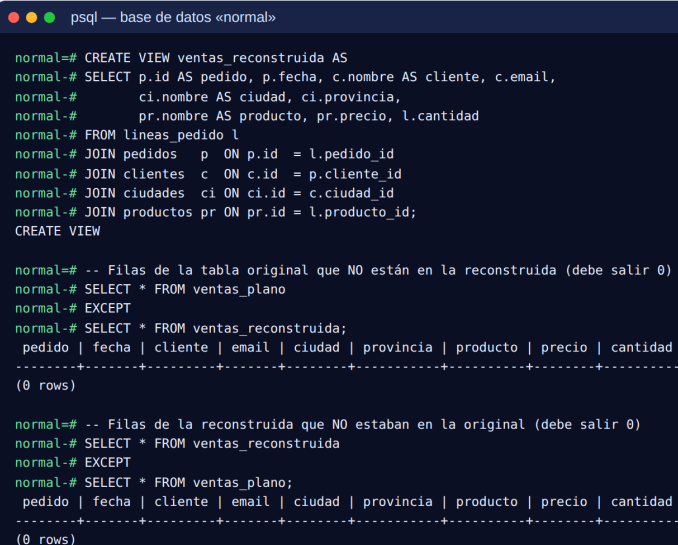
Figura 4.4-d. Captura real: rellenar las tablas definitivas a partir de la tabla original.

Qué ves en la imagen. Cada `INSERT ... SELECT` copia los datos que correspondan: `INSERT 0 3` (tres ciudades), `INSERT 0 3` (tres clientes), `INSERT 0 4` (cuatro pedidos) e `INSERT 0 5` (cinco líneas). Los `JOIN` convierten nombres en números de enlace (por ejemplo, el nombre de la ciudad en su `ciudad_id`). Los dos `SELECT` muestran las ciudades y clientes: cada uno aparece una sola vez.

```
CREATE VIEW ventas_reconstruida AS
SELECT p.id AS pedido, p.fecha, c.nombre AS cliente, c.email,
       ci.nombre AS ciudad, ci.provincia,
       pr.nombre AS producto, pr.precio, l.cantidad
FROM lineas_pedido l
JOIN pedidos    p ON p.id = l.pedido_id
JOIN clientes   c ON c.id = p.cliente_id
JOIN ciudades   ci ON ci.id = c.ciudad_id
JOIN productos pr ON pr.id = l.producto_id;

-- Filas de la tabla original que NO están en la reconstruida (debe salir 0)
SELECT * FROM ventas_plano
EXCEPT
SELECT * FROM ventas_reconstruida;

-- Filas de la reconstruida que NO estaban en la original (debe salir 0)
SELECT * FROM ventas_reconstruida
EXCEPT
SELECT * FROM ventas_plano;
```



```
psql — base de datos «normal»

normal=# CREATE VIEW ventas_reconstruida AS
normal=# SELECT p.id AS pedido, p.fecha, c.nombre AS cliente, c.email,
normal=#         ci.nombre AS ciudad, ci.provincia,
normal=#         pr.nombre AS producto, pr.precio, l.cantidad
normal=# FROM lineas_pedido l
normal=# JOIN pedidos    p ON p.id = l.pedido_id
normal=# JOIN clientes   c ON c.id = p.cliente_id
normal=# JOIN ciudades   ci ON ci.id = c.ciudad_id
normal=# JOIN productos pr ON pr.id = l.producto_id;
CREATE VIEW

normal=# -- Filas de la tabla original que NO están en la reconstruida (debe salir 0)
normal=# SELECT * FROM ventas_plano
normal=# EXCEPT
normal=# SELECT * FROM ventas_reconstruida;
pedido | fecha | cliente | email | ciudad | provincia | producto | precio | cantidad
-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

normal=# -- Filas de la reconstruida que NO estaban en la original (debe salir 0)
normal=# SELECT * FROM ventas_reconstruida
normal=# EXCEPT
normal=# SELECT * FROM ventas_plano;
pedido | fecha | cliente | email | ciudad | provincia | producto | precio | cantidad
-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)
```

Figura 4.4-e. Captura real: la prueba de que no se ha perdido información.

Qué ves en la imagen. La vista `ventas_reconstruida` vuelve a unir las cinco tablas para obtener la hoja original. `EXCEPT` devuelve las filas del primer resultado que **no** están en el segundo. Las dos comparaciones (original menos reconstruida y reconstruida menos original)

devuelven **0 filas**: las dos tablas contienen exactamente los mismos datos. La normalización ha reorganizado la información sin perder ni inventar nada.

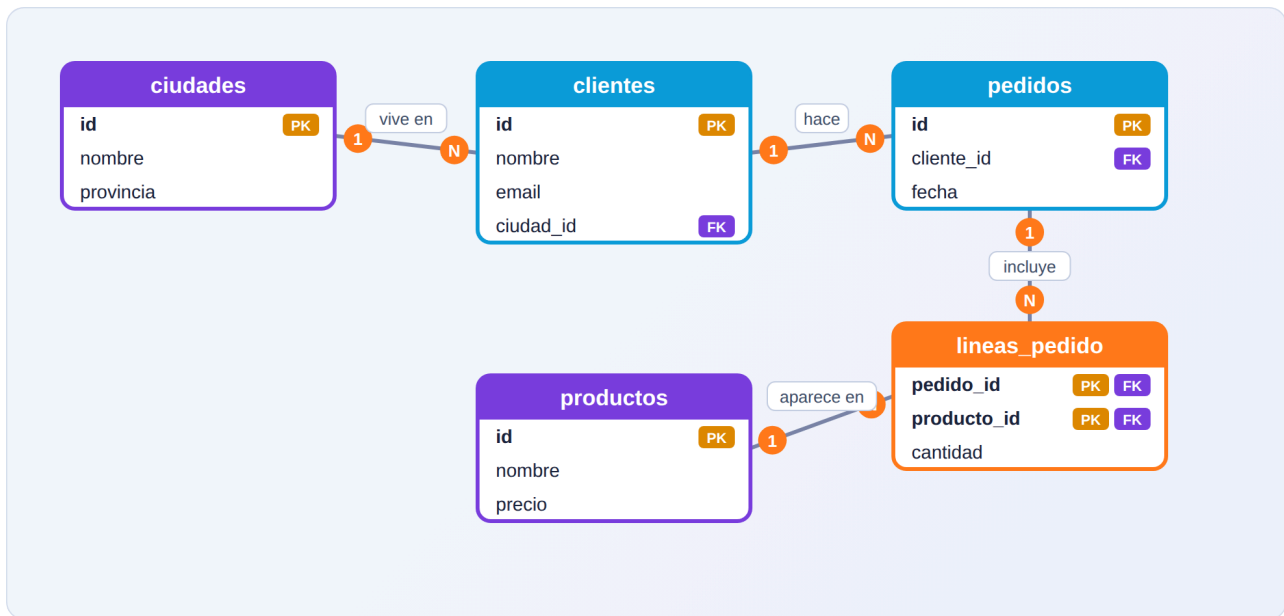


Figura 4.4-f. Diagrama del diseño final, generado desde PostgreSQL.

Qué ves en la imagen. Cinco tablas bien repartidas: `ciudades`, `clientes`, `pedidos`, `lineas_pedido` y `productos`, enlazadas con claves extranjeras.

```

UPDATE clientes SET email = 'ana.nueva@ejemplo.com' WHERE nombre = 'Ana García';

SELECT DISTINCT cliente, email FROM ventas_reconstruida WHERE cliente = 'Ana García';

DELETE FROM lineas_pedido WHERE pedido_id = 104;
DELETE FROM pedidos WHERE id = 104;

SELECT nombre FROM clientes ORDER BY nombre;

INSERT INTO productos (nombre, precio) VALUES ('Taza', 7.50);

SELECT nombre, precio FROM productos ORDER BY id;
  
```

```

psql — base de datos «normal»

normal=# UPDATE clientes SET email = 'ana.nueva@ejemplo.com' WHERE nombre = 'Ana García';
UPDATE 1

normal=# SELECT DISTINCT cliente, email FROM ventas_reconstruida WHERE cliente = 'Ana García';
 cliente | email
-----+-----
 Ana García | ana.nueva@ejemplo.com
(1 row)

normal=# DELETE FROM lineas_pedido WHERE pedido_id = 104;
normal=# DELETE FROM pedidos WHERE id = 104;
DELETE 1
DELETE 1

normal=# SELECT nombre FROM clientes ORDER BY nombre;
 nombre
-----
 Ana García
 Luis Pérez
 Marta Ruiz
(3 rows)

normal=# INSERT INTO productos (nombre, precio) VALUES ('Taza', 7.50);
INSERT 0 1

normal=# SELECT nombre, precio FROM productos ORDER BY id;
 nombre | precio
-----+-----
 Camiseta | 19.95
 Gorra | 9.90
 Mochila | 34.50
 Taza | 7.50
(4 rows)

```

Figura 4.4-g. Captura real: las tres anomalías, resueltas.

Qué ves en la imagen.

1. **Modificación:** el email de Ana se cambia en **una sola fila** de `clientes` (`UPDATE 1`); la vista muestra un único email para Ana en todos sus pedidos.
2. **Borrado:** se borran el pedido 104 y su línea. La lista de clientes sigue teniendo a **Luis Pérez**: ya no desaparece, porque el cliente es una fila propia.
3. **Insertión:** se da de alta el producto «Taza» sin ningún pedido (`INSERT 0 1`) y aparece en `productos`.

Ejemplo sencillo

La provincia depende de la ciudad, no del cliente. Si la guardas en la tabla de clientes, repites «Madrid» para cada cliente de Alcalá. En una tabla `ciudades`, está una vez.

Ejemplo real

Una empresa guardaba en cada factura el nombre y la dirección del cliente. Cuando un cliente cambió de dirección fiscal, las facturas antiguas quedaron con la dirección vieja (que, en este caso, sí era correcto conservar, punto 4.6) pero las nuevas tenían direcciones distintas entre sí. Separar el cliente permitió decidir qué se conserva y qué se actualiza.

Profundizando

Otras formas normales. Existen formas más estrictas: la forma normal de Boyce-Codd (BCNF), la cuarta y la quinta. En la práctica, casi todos los diseños profesionales se detienen en la 3FN (o

BCNF), que elimina la mayoría de los problemas reales.

Orden de rellenado. Al migrar datos, se llenan primero las tablas de las que otras dependen (ciudades, productos), después las que apuntan a ellas (clientes, pedidos) y al final las tablas intermedias.

Tablas temporales de migración. Las tablas `pedidos_2fn` y `lineas_2fn` eran solo un paso intermedio: una vez migrados los datos, se pueden borrar (`DROP TABLE`).

Las vistas reconstruyen la «hoja». Una vista como `ventas_reconstruida` permite que las aplicaciones o informes sigan viendo los datos como una tabla plana sin duplicarlos.

Ejercicio

1. En `empleados(id, nombre, departamento_id, nombre_departamento)`, ¿qué columna viola la 3FN y por qué?
2. Escribe las dos tablas resultantes con sus claves.
3. Comprueba con una consulta que, en `pedidos_2fn`, cada cliente tiene un solo email.

Solución

1. `nombre_departamento`: depende de `departamento_id`, que no es la clave de la tabla (la clave es `id`).
2. Por ejemplo:

```
CREATE TABLE departamentos (
  id      integer PRIMARY KEY,
  nombre text NOT NULL
);
CREATE TABLE empleados (
  id          integer PRIMARY KEY,
  nombre      text NOT NULL,
  departamento_id integer NOT NULL REFERENCES departamentos (id)
);
```

1. `SELECT cliente FROM pedidos_2fn GROUP BY cliente HAVING count(DISTINCT email) > 1;` devuelve 0 filas.

Error habitual

Separar tablas y olvidar enlazarlas con claves extranjeras, o rellenarlas en orden incorrecto y fallar por claves que no existen.

```
-- Mal: se separa la ciudad pero no se enlaza
CREATE TABLE clientes_mal (id integer PRIMARY KEY, nombre text, ciudad_nombre text);
-- Bien: se enlaza con una clave extranjera
CREATE TABLE clientes_bien (id integer PRIMARY KEY, nombre text, ciudad_id integer REFERENCES ciudades (id));
```

Buena práctica

Después de normalizar, **demuestra que no has perdido información** reconstruyendo la tabla original y comparándola con **EXCEPT**.

Comprobación

Para cada tabla pregunta: ¿cada dato depende de la clave, de toda la clave y de nada más que la clave? Si alguno depende de otra columna, hay una tabla escondida.

4.5 Cómo reconocer el problema en una tabla real

¿Qué es?

Es el método para mirar una tabla que no has diseñado tú (o que has diseñado hace tiempo) y averiguar si necesita normalizarse, y cómo.

¿Para qué sirve?

En la vida real casi nunca partes de cero: heredas hojas de cálculo, tablas de otros proyectos y bases de datos antiguas. Saber diagnosticarlas es una habilidad de todos los días.

¿Por qué lo necesito?

Porque los problemas de diseño no avisan. Hay que ir a buscarlos con preguntas concretas y comprobarlos con datos reales.

¿Cómo funciona?

Primero, **señales de alerta** en el diseño:

- Columnas con números en el nombre (`telefono1` , `producto2`).
- Listas o separadores dentro de una celda.
- Los mismos valores repetidos en muchas filas (nombres, emails, ciudades).
- Columnas casi siempre vacías.
- Nombres del tipo «datos del cliente» dentro de una tabla de pedidos.
- Cambiar un dato exige modificar muchas filas.

Después, **comprobar dependencias con consultas**. Para saber si «A determina B»: agrupa por A y comprueba si B toma más de un valor.

```
SELECT A FROM tabla GROUP BY A HAVING count(DISTINCT B) > 1;
```

Si la consulta devuelve **0 filas**, cada valor de A tiene un único valor de B en los datos actuales: la dependencia `A → B` se cumple. Si devuelve filas, esas filas son las excepciones: no hay dependencia.

Primero, sin código: comprobar una dependencia a mano

1

Pregunta a mano «¿A determina B?»: ordena por A y mira si B cambia

ordenada por producto	
producto	precio
Camiseta	19.95
Camiseta	19.95
Gorra	9.90
Gorra	9.90
Mochila	34.50

ordenada por provincia	
provincia	cliente
Madrid	Ana García
Madrid	Ana García
Madrid	Marta Ruiz
Sevilla	Luis Pérez

Izquierda: cada producto siempre tiene el mismo precio → producto determina precio (repetido sin necesidad). Derecha: Madrid tiene dos clientes distintos → provincia NO determina cliente.

2

Cuidado: con pocos datos puede haber coincidencias

Si en la tabla solo hay un cliente por ciudad, «ciudad determina cliente» parece cumplirse. Pero **nada en el negocio** impide que dos clientes vivan en la misma ciudad.

Una dependencia es una regla del negocio, no una coincidencia de los datos.

Por eso las dependencias se deciden pensando en el negocio, y los datos de ejemplo solo sirven para comprobarlas.

Figura 4.5-a. Sin código: cómo comprobar una dependencia ordenando la tabla.

Qué ves en la imagen. Paso 1: ordenando por producto, el precio es siempre el mismo dentro de cada producto (producto determina precio). Ordenando por provincia, Madrid tiene dos clientes distintos (provincia no determina cliente). Paso 2: la advertencia más importante: **una dependencia es una regla del negocio, no una coincidencia de los datos**. Si en tu muestra solo hay un cliente por ciudad, «ciudad determina cliente» parece cumplirse, pero nada en el negocio lo garantiza.

Paso a paso: ahora con código

1. **Elige una pareja de columnas** que sospechas dependientes (A, B).
2. **Ejecuta la consulta** de comprobación `GROUP BY A HAVING count(DISTINCT B) > 1`.
3. **Interpreta**: 0 filas significa que se cumple en tus datos; filas significan que no.
4. **Contrasta con el negocio**: ¿tiene sentido que A determine B siempre? Si solo se cumple por casualidad, no la uses para normalizar.
5. **Repite** con otras parejas.

Código y resultado

```
-- ¿Se cumple «producto determina precio»? 0 filas = sí, hay dependencia
SELECT producto, count(DISTINCT precio) AS precios_distintos
FROM ventas_plano
GROUP BY producto
HAVING count(DISTINCT precio) > 1;

-- ¿Se cumple «pedido determina fecha»?
SELECT pedido, count(DISTINCT fecha) AS fechas_distintas
FROM ventas_plano
GROUP BY pedido
HAVING count(DISTINCT fecha) > 1;

-- ¿Y «provincia determina cliente»? (si salen filas, NO hay dependencia)
SELECT provincia, count(DISTINCT cliente) AS clientes_distintos
FROM ventas_plano
GROUP BY provincia
HAVING count(DISTINCT cliente) > 1;

-- Cuidado: «ciudad determina cliente» parece cumplirse... ¡solo por casualidad!
SELECT ciudad, count(DISTINCT cliente) AS clientes_distintos
FROM ventas_plano
GROUP BY ciudad
HAVING count(DISTINCT cliente) > 1;
```

```
psql — base de datos «normal»

normal=# -- ¿Se cumple «producto determina precio»? 0 filas = sí, hay dependencia
normal=# SELECT producto, count(DISTINCT precio) AS precios_distintos
normal=# FROM ventas_plano
normal=# GROUP BY producto
normal=# HAVING count(DISTINCT precio) > 1;
 producto | precios_distintos 
-----+-----
(0 rows)

normal=# -- ¿Se cumple «pedido determina fecha»?
normal=# SELECT pedido, count(DISTINCT fecha) AS fechas_distintas
normal=# FROM ventas_plano
normal=# GROUP BY pedido
normal=# HAVING count(DISTINCT fecha) > 1;
 pedido | fechas_distintas 
-----+-----
(0 rows)

normal=# -- ¿Y «provincia determina cliente»? (si salen filas, NO hay dependencia)
normal=# SELECT provincia, count(DISTINCT cliente) AS clientes_distintos
normal=# FROM ventas_plano
normal=# GROUP BY provincia
normal=# HAVING count(DISTINCT cliente) > 1;
 provincia | clientes_distintos 
-----+-----
 Madrid   |                2 
(1 row)

normal=# -- Cuidado: «ciudad determina cliente» parece cumplirse... ¡solo por casualidad!
normal=# SELECT ciudad, count(DISTINCT cliente) AS clientes_distintos
normal=# FROM ventas_plano
normal=# GROUP BY ciudad
normal=# HAVING count(DISTINCT cliente) > 1;
 ciudad | clientes_distintos 
-----+-----
(0 rows)
```

Figura 4.5-b. Captura real: cuatro comprobaciones de dependencia, tres que se cumplen y una que solo coincide por casualidad.

Qué ves en la imagen.

1. `producto → precio` : 0 filas. Se cumple, y tiene sentido de negocio: un producto tiene un precio.
2. `pedido → fecha` : 0 filas. Se cumple: un pedido tiene una fecha.
3. `provincia → cliente` : devuelve filas. **No hay dependencia**: Madrid tiene dos clientes distintos, así que no se puede determinar el cliente a partir de la provincia.
4. `ciudad → cliente` : devuelve 0 filas. Los datos *parecen* cumplirla, pero **es pura casualidad de la muestra**: en la tienda real, dos clientes pueden vivir en Alcalá. Por eso la última consulta lleva el aviso: una dependencia se decide con el negocio, y los datos solo la confirman o la refutan.

Ejemplo sencillo

En una hoja de cálculo de alumnos, detectas que el nombre del curso aparece repetido para cada alumno. La pregunta `curso → nombre_curso` da 0 filas: hay que sacar una tabla `cursos`.

Ejemplo real

Antes de migrar una base de datos heredada, un equipo ejecuta decenas de comprobaciones de dependencias sobre las tablas principales para decidir qué tablas nuevas crear. Es el primer paso de cualquier proyecto de rediseño.

Profundizando

Falsos positivos y falsos negativos. Con pocos datos, una dependencia falsa puede parecer cierta (como `ciudad → cliente`). Con datos sucios, una dependencia verdadera puede parecer falsa (por ejemplo, «Madrid» y «madrid» cuentan como valores distintos).

Datos sucios. Si una comprobación devuelve excepciones inesperadas (el mismo producto con dos precios), puede ser un error de datos que conviene investigar antes de normalizar.

Herramientas. Algunos programas de análisis de datos detectan dependencias automáticamente, pero la decisión final sigue siendo tuya y del negocio.

Ejercicio

1. Escribe la consulta que comprueba si «cada cliente tiene una sola ciudad» en `ventas_plano`.
2. En una tabla de empleados, sospechas que `departamento → jefe`. ¿Cómo lo comprobarías y cómo lo contrastarías con el negocio?
3. Si la consulta de comprobación devuelve 2 filas, ¿qué significa?

Solución

1. `SELECT cliente FROM ventas_plano GROUP BY cliente HAVING count(DISTINCT ciudad) > 1;`
2. Con `SELECT departamento FROM empleados GROUP BY departamento HAVING count(DISTINCT jefe) > 1;`. Con el negocio: ¿un departamento puede tener dos jefes a la

vez? Si sí, no hay dependencia.

3. Que hay 2 valores de A con más de un valor de B: esas son las excepciones, y la dependencia no se cumple (o hay datos erróneos).

Error habitual

Fiarse solo de los datos: normalizar basándose en una dependencia que solo se cumple por casualidad en una muestra pequeña.

```
-- Parece que ciudad determina cliente... pero es casualidad de la muestra
SELECT ciudad FROM ventas_plano GROUP BY ciudad HAVING count(DISTINCT cliente) >
1;
```

Buena práctica

Documenta cada dependencia que uses para normalizar, con su justificación de negocio, no solo «los datos lo cumplen».

Comprobación

Para una tabla tuya, escribe tres dependencias, comprueba cada una con la consulta y justifícala con una frase de negocio.

4.6 Cuándo normalizar y cuándo desnormalizar

¿Qué es?

Normalizar es repartir los datos para que cada uno viva en un solo sitio. **Desnormalizar** es repetir un dato **a propósito**, sabiendo por qué y cómo se protege.

¿Para qué sirve?

Para saber elegir. La regla general es normalizar siempre; las excepciones son pocas y deben estar justificadas.

¿Por qué lo necesito?

Un diseño totalmente normalizado necesita muchos **JOIN** para leer cosas simples, y a veces hay consultas críticas que tardan demasiado. Pero desnormalizar sin criterio vuelve a traer las anomalías del punto 4.1.

¿Cómo funciona?

Hay dos preguntas que justifican repetir un dato:

1. **¿Es un dato histórico?** (el precio al que se vendió, el nombre en una factura emitida): se repite porque debe conservar lo que era cierto en ese momento.
2. **¿Es un dato calculado para leer más rápido?** (el total de un pedido): se repite por rendimiento, y entonces hay que **mantenerlo coherente**.

Si ninguna es cierta, **no desnormalices**.

Primero, sin código: casos habituales

Caso	¿Por qué desnormalizar?	Riesgo
Guardar el precio de venta en la línea de pedido	El precio del producto cambia; el pedido debe conservar el de su día	Ninguno: es un dato propio
Guardar el total del pedido en la tabla pedidos	Leer el total sin sumar las líneas cada vez	Que el total no coincida con las líneas
Tabla de resumen para informes (ventas por mes)	Los informes tardan menos	Datos desactualizados
Copiar el nombre del cliente en la factura emitida	La factura debe mostrar lo que era cierto el día de emisión	Ninguno si se trata como dato histórico

Ilustración: cuándo puede tener sentido desnormalizar y cómo hacerlo con cuidado.

Figura 4.6-a. Casos en los que puede tener sentido desnormalizar, su riesgo y cómo protegerlo.

Qué ves en la imagen. Cuatro casos reales: el precio de venta en la línea (histórico, sin riesgo), el total en el pedido (riesgo: que no coincida con las líneas), una tabla de resumen para informes (riesgo: datos desactualizados) y la copia del nombre en una factura (histórico, correcto). Cada uno con la forma de protegerlo.

Paso a paso: ahora con código

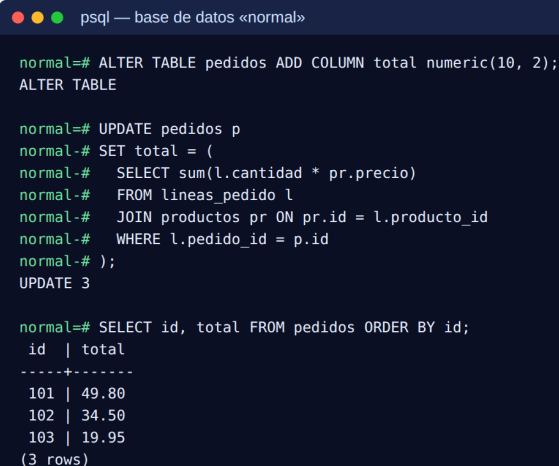
1. **Añade una columna `total`** a `pedidos` (dato calculado, redundante a propósito).
2. **Rellénala** calculando la suma de las líneas.
3. **Lee el total** sin hacer ningún `JOIN`: es el beneficio.
4. **Provoca el riesgo**: cambia una línea y observa que el total guardado queda desactualizado.
5. **Detéctalo con una consulta** que compare el total guardado con el recalculado.
6. **Decide cómo mantenerlo coherente** (recalcular al cambiar una línea; Parte 5 y 6: triggers y vistas materializadas).

Código y resultado

```
ALTER TABLE pedidos ADD COLUMN total numeric(10, 2);

UPDATE pedidos p
SET total = (
  SELECT sum(l.cantidad * pr.precio)
  FROM lineas_pedido l
  JOIN productos pr ON pr.id = l.producto_id
  WHERE l.pedido_id = p.id
);

SELECT id, total FROM pedidos ORDER BY id;
```



```
psql — base de datos «normal»

normal=# ALTER TABLE pedidos ADD COLUMN total numeric(10, 2);
ALTER TABLE

normal=# UPDATE pedidos p
normal=# SET total = (
normal=#   SELECT sum(l.cantidad * pr.precio)
normal=#   FROM lineas_pedido l
normal=#   JOIN productos pr ON pr.id = l.producto_id
normal=#   WHERE l.pedido_id = p.id
normal=# );
UPDATE 3

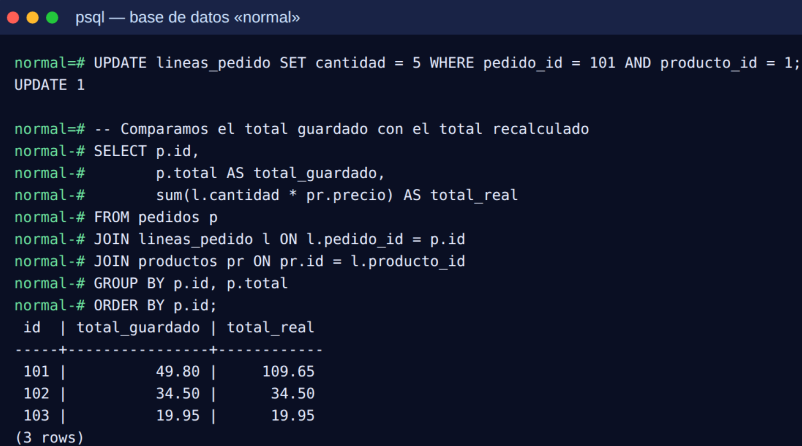
normal=# SELECT id, total FROM pedidos ORDER BY id;
 id | total
-----+-----
 101 | 49.80
 102 | 34.50
 103 | 19.95
(3 rows)
```

Figura 4.6-b. Captura real: guardar el total del pedido como dato calculado.

Qué ves en la imagen. `ALTER TABLE ... ADD COLUMN total` añade la columna. El `UPDATE` rellena cada pedido con la suma de `cantidad × precio` de sus líneas, usando una subconsulta. `UPDATE 3` indica que se actualizaron tres pedidos. Ahora `SELECT id, total FROM pedidos` da el total sin ninguna unión. (El pedido 101: $2 \times 19,95 + 1 \times 9,90 = 49,80$.)

```
UPDATE lineas_pedido SET cantidad = 5 WHERE pedido_id = 101 AND producto_id = 1;

-- Comparamos el total guardado con el total recalculado
SELECT p.id,
       p.total AS total_guardado,
       sum(l.cantidad * pr.precio) AS total_real
FROM pedidos p
JOIN lineas_pedido l ON l.pedido_id = p.id
JOIN productos pr ON pr.id = l.producto_id
GROUP BY p.id, p.total
ORDER BY p.id;
```



```
psql — base de datos «normal»

normal=# UPDATE lineas_pedido SET cantidad = 5 WHERE pedido_id = 101 AND producto_id = 1;
UPDATE 1

normal=# -- Comparamos el total guardado con el total recalculado
normal=# SELECT p.id,
normal=#         p.total AS total_guardado,
normal=#         sum(l.cantidad * pr.precio) AS total_real
normal=# FROM pedidos p
normal=# JOIN lineas_pedido l ON l.pedido_id = p.id
normal=# JOIN productos pr ON pr.id = l.producto_id
normal=# GROUP BY p.id, p.total
normal=# ORDER BY p.id;
 id | total_guardado | total_real 
-----+-----+-----
 101 |         49.80 |        109.65
 102 |         34.50 |         34.50
 103 |         19.95 |         19.95
(3 rows)
```

Figura 4.6-c. Captura real: el riesgo. El total guardado ya no coincide con las líneas.

Qué ves en la imagen. Se cambia la cantidad de camisetas del pedido 101 de 2 a 5. La consulta compara el `total_guardado` con el `total_real` (recalculado sumando las líneas): el pedido 101 tiene guardado 49,80 pero el total real es 109,65. Los otros dos pedidos siguen coincidiendo. Esta consulta de control es la forma de **detectar** que la redundancia se ha desincronizado.

Ejemplo sencillo

Guardar `precio_venta` en la línea de pedido: es un dato histórico, no un error.

Ejemplo real

Una tienda con 50 millones de pedidos guarda el total en cada uno porque calcularlo sumando las líneas, cada vez que se muestra un listado, era demasiado lento. Un proceso recalcula el total cada vez que cambia una línea y otro lo verifica cada noche.

Profundizando

Mide antes de desnormalizar. Con los índices adecuados (Parte 18), muchas consultas con `JOIN` son muy rápidas. Desnormaliza solo después de medir un problema real de rendimiento.

Mantener la coherencia. Hay tres mecanismos principales: **triggers** (código que se ejecuta automáticamente al cambiar una línea), **vistas materializadas** (resultados guardados que se refrescan) y **la propia aplicación** (que recalcula). Cada uno tiene ventajas y riesgos; se ven en las Partes 5 y 6.

Columnas generadas. PostgreSQL permite columnas que se calculan automáticamente a partir de otras columnas **de la misma fila** (por ejemplo, un importe con IVA). No sirven para totales de varias filas.

Desnormalizar para análisis. En almacenes de datos para informes (*data warehouses*), la desnormalización es la norma por diseño: se lee mucho y se escribe poco.

Ejercicio

1. Clasifica estos datos como «histórico», «calculado» o «error»: el precio de venta en cada línea, el total del pedido, el nombre del cliente copiado en cada pedido sin motivo.
2. ¿Cómo detectarías pedidos cuyo total guardado no coincide con sus líneas?
3. ¿Qué mecanismo usarías para actualizar el total automáticamente al cambiar una línea?

Solución

1. Precio de venta: histórico. Total del pedido: calculado. Nombre del cliente copiado sin motivo: error (redundancia accidental).
2. Con una consulta que compare `total` con `sum(cantidad * precio)` agrupada por pedido (como la figura 4.6-c).
3. Un trigger sobre `lineas_pedido` que recalcule el total del pedido afectado, o un recálculo desde la aplicación dentro de la misma transacción (Partes 5 y 6).

Error habitual

Desnormalizar sin ningún mecanismo de control: nadie comprueba ni recalcula, y los datos se desincronizan en silencio.

```
-- Mal: dato calculado sin nada que lo mantenga ni lo compruebe
ALTER TABLE pedidos ADD COLUMN total numeric(10, 2);
```

Buena práctica

Todo dato repetido a propósito debe tener: (1) una razón escrita, (2) una forma de mantenerlo coherente y (3) una consulta que compruebe que lo es.

Comprobación

Para cada dato duplicado de tu diseño, responde: ¿es histórico o calculado? ¿Cómo lo mantengo? ¿Cómo lo compruebo? Si no tienes respuesta a las tres, quítalo.

4.7 Errores habituales de diseño

¿Qué es?

Son los errores de diseño que más se repiten en proyectos reales. Reconocerlos ahorra mucho trabajo.

¿Para qué sirve?

Para detectar los problemas cuando todavía es barato corregirlos: en el papel, no con millones de filas.

¿Por qué lo necesito?

Casi todos los problemas que verás en bases de datos heredadas son variantes de los mismos cinco o seis errores.

¿Cómo funciona?

Los errores principales:

Error	Síntoma	Solución
Columnas repetidas	producto1 , producto2 ...	Tabla aparte con una fila por elemento
Listas dentro de una celda	«Camiseta, Gorra»	Una fila por valor (1FN)
Mezclar entidades	Cliente, producto y pedido en una tabla	Una tabla por entidad
Tabla «clave-valor» para todo (EAV)	(entidad, atributo, valor)	Columnas reales con tipos y reglas
Datos calculados sin control	Totales desactualizados	Mecanismo de actualización y comprobación (4.6)
Normalizar de más	Decenas de tablas de una columna	Mantener juntos los datos que siempre se usan juntos y no se repiten

Primero, sin código: reconocerlos

✗ Clave-valor para todo (EAV)

datos	
entidad_id	atributo
1	nombre
1	edad

Sin tipos ni reglas: «treinta» es una edad válida. Las consultas se vuelven complicadas.

✗ Columnas repetidas

pedidos		
id	producto1	pr
101	Camiseta	Go

¿Y si hay cuatro productos? Hay que cambiar la tabla entera.

✗ Mezclar entidades

ventas		
pedido	cliente	em
101	Ana	an

Cliente, producto y pedido en una sola tabla: todos los problemas del punto 4.1.

✗ Lista dentro de una celda

contactos		
id	nombre	telefono
1	Ana	600111

No se puede buscar ni validar cada teléfono.

Ilustración: cuatro anti-patrones habituales.

Figura 4.7-a. Sin código: cuatro anti-patrones habituales.

Qué ves en la imagen. Cuatro tarjetas con un ejemplo y su problema: la tabla clave-valor (sin tipos ni reglas), las columnas repetidas, la mezcla de entidades y la lista en una celda.

Paso a paso: ahora con código

1. **Crea una tabla clave-valor** (el anti-patrón EAV) y guarda en ella los datos de dos personas.
2. **Intenta obtener una fila por persona:** observa lo complicada que es la consulta.
3. **Comprueba que no hay reglas:** guarda una edad como texto («treinta») y mira que se acepta.
4. **Compara con una tabla normal** con columnas, tipos y restricciones.

Código y resultado

```
-- Anti-patrón: tabla «clave-valor» para guardarlo todo
CREATE TABLE datos (
  entidad_id integer,
  atributo   text,
  valor      text
);

INSERT INTO datos VALUES
  (1, 'nombre', 'Ana García'), (1, 'email', 'ana@ejemplo.com'), (1, 'edad',
  'treinta'),
  (2, 'nombre', 'Luis Pérez'), (2, 'email', 'luis@ejemplo.com');

-- Para ver nombre y email en una fila hay que desmontar y volver a montar
SELECT entidad_id,
       max(valor) FILTER (WHERE atributo = 'nombre') AS nombre,
       max(valor) FILTER (WHERE atributo = 'email')  AS email
FROM datos
GROUP BY entidad_id
ORDER BY entidad_id;

-- Y nada impide guardar disparates: la edad es texto
SELECT valor FROM datos WHERE atributo = 'edad';
```

```
psql — base de datos «normal»

normal=# -- Anti-patrón: tabla «clave-valor» para guardarlo todo
normal=# CREATE TABLE datos (
normal=#   entidad_id integer,
normal=#   atributo text,
normal=#   valor text
normal=# );
CREATE TABLE

normal=# INSERT INTO datos VALUES
normal=#   (1, 'nombre', 'Ana García'), (1, 'email', 'ana@ejemplo.com'), (1, 'edad', 'treinta'),
normal=#   (2, 'nombre', 'Luis Pérez'), (2, 'email', 'luis@ejemplo.com');
INSERT 0 5

normal=# -- Para ver nombre y email en una fila hay que desmontar y volver a montar
normal=# SELECT entidad_id,
normal=#       max(valor) FILTER (WHERE atributo = 'nombre') AS nombre,
normal=#       max(valor) FILTER (WHERE atributo = 'email') AS email
normal=# FROM datos
normal=# GROUP BY entidad_id
normal=# ORDER BY entidad_id;
entidad_id | nombre | email
-----+-----+-----
1 | Ana García | ana@ejemplo.com
2 | Luis Pérez | luis@ejemplo.com
(2 rows)

normal=# -- Y nada impide guardar disparates: la edad es texto
normal=# SELECT valor FROM datos WHERE atributo = 'edad';
valor
-----
treinta
(1 row)
```

Figura 4.7-b. Captura real: el anti-patrón clave-valor (EAV). Todo cabe, pero nada se protege.

Qué ves en la imagen. La tabla `datos` guarda cada atributo de cada entidad como una fila (`entidad_id`, `atributo`, `valor`). Parece flexible, pero: para ver el nombre y el email en una sola fila hay que usar `max(...) FILTER (WHERE ...)` y `GROUP BY` (una consulta complicada que crece con cada atributo); `valor` es siempre texto, así que la edad «treinta» se acepta sin problema; no hay `NOT NULL`, ni `UNIQUE`, ni claves extranjeras; y el gestor no puede ayudarte a validar nada. Una tabla normal con las columnas `nombre`, `email` y `edad integer` resuelve todo esto.

Ejemplo sencillo

Una tabla `pedidos` con columnas `producto1`, `producto2` y `producto3`: ¿qué pasa con el cuarto producto?

Ejemplo real

Un sistema de «formularios configurables» se montó con una tabla clave-valor para todo. Un año después, los informes más sencillos tardaban minutos y había edades como «n/a», «treinta» y «-1» en la misma columna.

Profundizando

Normalizar de más. Separar cada columna en su propia tabla (una tabla de nombres, otra de apellidos...) complica las consultas sin eliminar ninguna redundancia real. La regla: separa cuando hay datos repetidos o una cosa con vida propia, no por sistema.

Cuando el esquema es realmente variable. Para datos que cambian mucho de forma y no se consultan por campo (configuraciones, formularios), PostgreSQL ofrece la columna `jsonb` (Parte

6), que da flexibilidad **con** soporte de consulta e índices. Es mejor que EAV, pero sigue sin darte tipos ni restricciones por campo, así que úsala con criterio.

Detectar el problema pronto. La señal más fiable es que las consultas sencillas se vuelven complicadas. Si leer algo obvio exige un `max(...) FILTER`, algo va mal en el diseño.

Ejercicio

1. Corrige este diseño: `facturas(id, cliente, direccion_cliente, producto1, precio1, producto2, precio2)`.
2. ¿Qué dos errores de la tabla anterior son de 1FN y cuál de 3FN?
3. Escribe una tabla normal para guardar nombre, email y edad (entera) con las restricciones adecuadas.

Solución

1. Cuatro tablas: `clientes(id, nombre, direccion)`, `productos(id, nombre, precio)`, `facturas(id, cliente_id)` y `lineas_factura(factura_id, producto_id, cantidad, precio_venta)`.
2. De 1FN: las columnas repetidas `producto1`, `precio1`, `producto2`, `precio2`. De 3FN: `direccion_cliente`, que depende del cliente y no de la factura.
3. Por ejemplo:

```
CREATE TABLE personas (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,
  email   text NOT NULL UNIQUE,
  edad    integer CHECK (edad BETWEEN 0 AND 130)
);
```

Error habitual

Usar la flexibilidad como excusa para no diseñar («guardo todo como clave-valor y así no tengo que cambiar nunca la estructura»). Se paga después, en consultas, rendimiento y calidad de datos.

```
-- Mal: nada impide guardar disparates
INSERT INTO datos VALUES (1, 'edad', 'treinta');
```

Buena práctica

Diseña columnas reales, con tipos y restricciones. Cambiar la estructura (añadir una columna) es fácil y está soportado; recuperar la calidad de datos perdida no.

Comprobación

Revisa tu diseño con la tabla de errores: ¿hay columnas repetidas, listas en celdas, entidades mezcladas, tablas clave-valor o datos calculados sin control?

Resumen de la Parte 4

- Redundancia es repetir datos; provoca anomalías de modificación, de inserción y de borrado.
- **1FN**: cada celda un solo valor y sin columnas repetidas; una fila por cada valor.
- **2FN**: con clave compuesta, cada dato depende de la clave completa; lo que depende de una parte va a su tabla.
- **3FN**: ningún dato depende de otro dato que no sea la clave; las cadenas $A \rightarrow B \rightarrow C$ se separan.
- La normalización no pierde información: se demuestra reconstruyendo la tabla original y comparando con `EXCEPT`.
- Las dependencias se comprueban con `GROUP BY ... HAVING count(DISTINCT ...) > 1`, pero se deciden con el negocio.
- Se normaliza por defecto; se desnormaliza solo con una razón (histórico o calculado) y con un mecanismo de control.
- Errores habituales: columnas repetidas, listas en celdas, entidades mezcladas, clave-valor para todo y normalizar de más.

Glosario de la Parte 4

Término	Significado
Redundancia	Guardar el mismo dato en más de un sitio
Anomalía	Problema que causa la redundancia (de modificación, de inserción o de borrado)
Normalización	Repartir los datos en tablas para eliminar la redundancia
Forma normal (1FN, 2FN, 3FN)	Cada una de las tres comprobaciones de un buen reparto
Dependencia	A determina B: sabiendo A, se sabe B
Dependencia transitiva	A determina B y B determina C: C depende de A «a través de» B
Valor atómico	Valor que no se puede dividir con sentido
Clave compuesta	Clave formada por varias columnas
Desnormalizar	Repetir un dato a propósito, con razón y control
EAV	Anti-patrón de tabla clave-valor (entidad, atributo, valor)
<code>EXCEPT</code>	Operación SQL que devuelve las filas de un resultado que no están en otro
<code>CREATE TABLE ... AS SELECT</code>	Crea una tabla a partir del resultado de una consulta

Mini examen de la Parte 4

1. ¿Qué es una anomalía de modificación? Pon un ejemplo.
2. ¿Qué dos señales indican que una tabla no cumple la 1FN?
3. ¿Cuándo tiene sentido hablar de 2FN?
4. ¿Qué es una dependencia transitiva? Pon un ejemplo.
5. ¿Cómo se comprueba con SQL que «A determina B»?
6. ¿Por qué una dependencia no debe decidirse solo con los datos?
7. ¿Cómo demuestras que no has perdido información al normalizar?
8. Da dos razones válidas para desnormalizar.
9. ¿Qué riesgo tiene guardar el total de un pedido y cómo lo detectas?
10. ¿Por qué la tabla clave-valor (EAV) suele ser un mal diseño?

Respuestas

1. Cambiar un dato repetido en un sitio y olvidar los demás, de modo que se contradicen (Ana con dos emails distintos).
2. Listas dentro de una celda, y columnas repetidas con un número (`telefono1` , `telefono2`).
3. Cuando la clave primaria está formada por varias columnas.
4. Cuando A determina B y B determina C: C depende de A a través de B. Ejemplo: pedido → cliente → ciudad.
5. Con `SELECT A FROM t GROUP BY A HAVING count(DISTINCT B) > 1`: 0 filas significa que se cumple en los datos.
6. Porque con pocos datos puede cumplirse por casualidad; es una regla del negocio.
7. Reconstruyendo la tabla original con una vista y comparando con `EXCEPT` en los dos sentidos: 0 filas en ambos.
8. Que el dato sea histórico (precio de venta) o calculado para leer más rápido (total del pedido), siempre con control.
9. Que el total guardado deje de coincidir con las líneas; se detecta comparándolo con el total recalculado.
10. Porque no hay tipos ni restricciones, las consultas se complican y la calidad de los datos se pierde.

Si has acertado 8 o más, estás listo para la Parte 5. Si no, repasa los puntos de las preguntas falladas.